The important point here is that the argument to Kredis.string is not the value of the string, it's the key that Redis uses to store the string. You can then use value= to set the string, and value to get the set value. Each of these triggers a call to Redis to manage the information.

Kredis defines the basic types boolean, datetime, decimal, float, integer, and string, all of which respond to value and value= and return the similarly named Ruby types (decimal corresponds to BigDecimal). There's also a Kredis.json type, which expects a Ruby hash and stores it in Redis as JSON.

Redis also defines more complex data structures that you can access via Kredis. I'm not going to go into the underlying Redis commands that the Kredis objects wrap, nor is this a complete list of the Kredis API; the documentation at https://github.com/rails/kredis covers that:

- Kredis.list(key): You add an element to the list with append or prepend, remove them with remove and get the elements out with elements. There is also Kredis.unique_list, which prevents elements from being duplicated.

- Kredis.set: A set differs from a list by being unordered. You can add or remove from a set and get the full set with members. Sets also have a fast include? method.

- Kredis.hash: You can add to the hash with update(key, value), access using regular hash square bracket syntax, and retrieve data with to_h, keys, and values.

All of these take an optional keyword argument, typed:, which takes one of the basic type names and ensures that all the data is retrieved as the expected type. Where appropriate you can set a default: keyword option. All the objects support an expires_in: option, which takes a value in seconds and automatically removes the data from Redis at the end of that time.

Redis has a few special data structures that are useful for specific kinds of data:

- Kredis.counter, which can accept increment or decrement and returns its current